Project Documentation

System Overview

This project implements a small VHDL-based classifier system on a Nexys A7 FPGA board. The system receives four binary input features from the FPGA switches `SW(3 downto 0)`. These inputs are used by a simple logistic regression-style classifier. The classifier calculates a weighted sum using predefined fixed weights and a bias value. After that, a threshold activation is applied to decide whether the result belongs to class 0 or class 1.

The classification result is displayed using `CLASS_LED`. The calculated score is displayed on the seven-segment display using the `SEG` and `AN` outputs.

The hardware system is divided into three main VHDL modules:

`control_unit → classifier → output_interface`

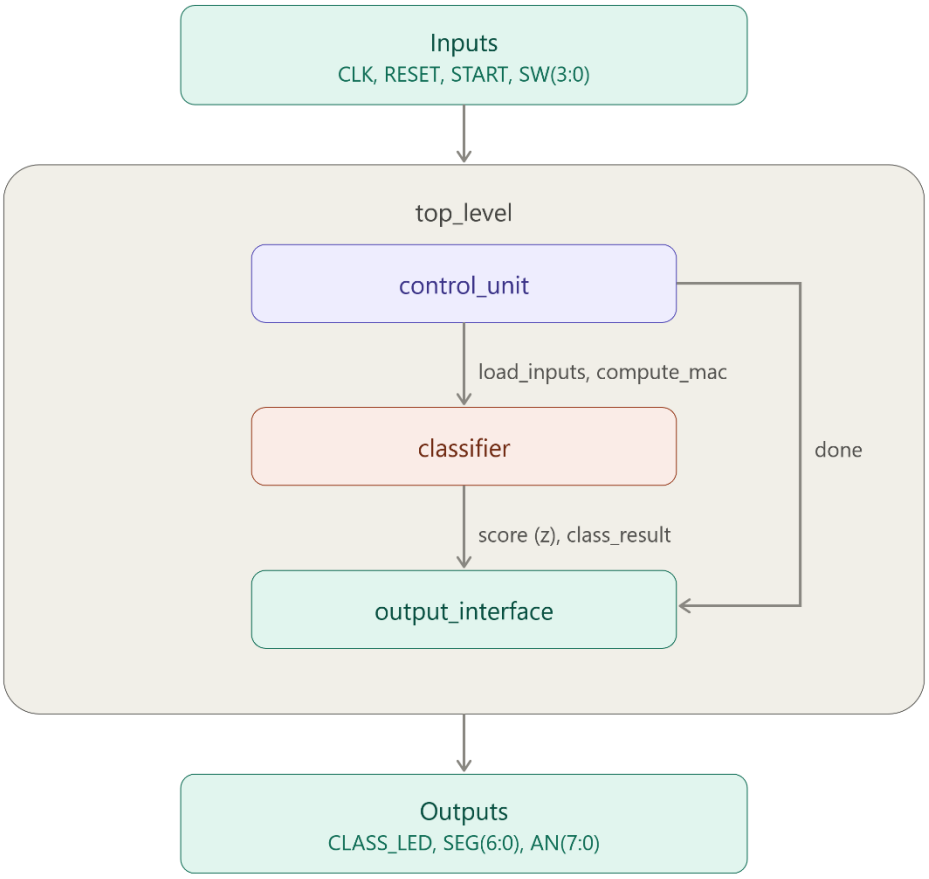
The `top_level` module connects these internal modules to the external FPGA board inputs and outputs.

Requirements

The requirements for this project are fairly straightforward. For hardware implementation, an FPGA such as the ***Nexys A7 FPGA board*** is required, for simulation and logic verification of the code QuestaSim or ModelSim, for synthesis and uploading the code to the FPGA board, tools such as Xilinx Vivado.

System Block Diagram

The system architecture consists of a control unit, a classifier and an output interface. The general data flow is shown below:



- **Architecture diagram**

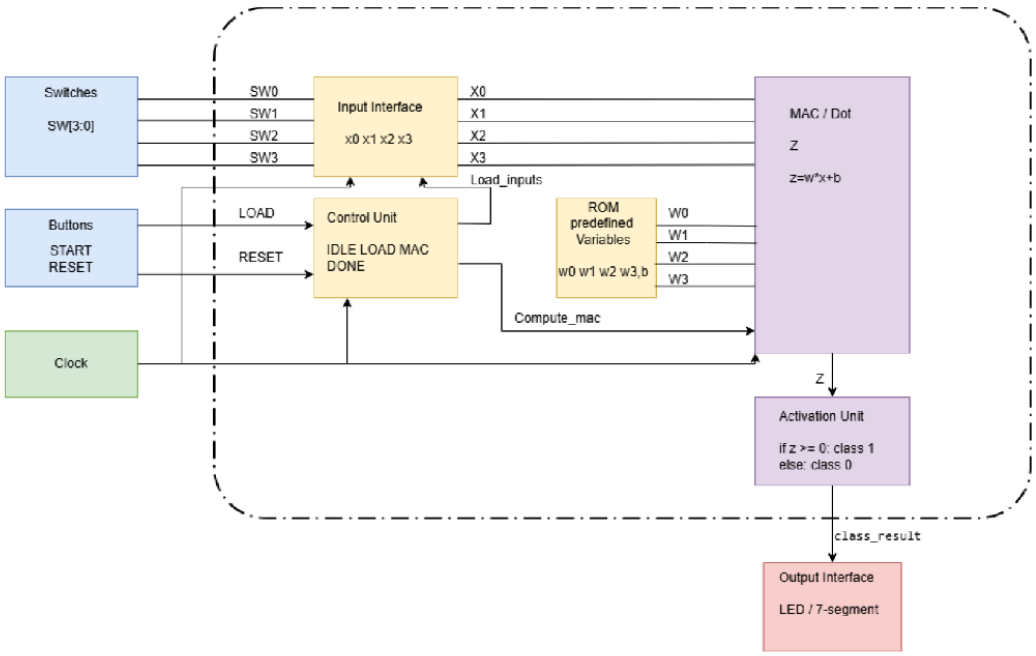


Figure: Architecture Diagram

The control_unit controls the timing of the classification process. The classifier stores the switch inputs and calculates the weighted sum. The output_interface stores the final result and displays it on the FPGA board.

Control Unit

The control_unit module is implemented as a finite state machine. It controls when the input values are loaded, when the weighted sum is calculated and when the result is marked as ready.

The state sequence is:

IDLE → LOAD → MAC → DONE_STATE → IDLE

In the IDLE state, the system waits for the START button. When START is active, the FSM moves to the LOAD state. In this state, the signal load_inputs is activated for one clock cycle. This tells the classifier to store the current switch values.

After that, the FSM moves to the MAC state. In this state, the signal compute_mac is activated, and the classifier calculates the weighted sum. Finally, the FSM enters the DONE_STATE, where the done signal is activated. This tells the output interface that the classification result is ready. After this state, the system returns to IDLE.

Classifier

The classifier module performs the main inference operation. It receives the four switch inputs SW(3 downto 0) as binary features. Internally, these inputs are stored as:

```
SW(0) = x0
SW(1) = x1
SW(2) = x2
SW(3) = x3
```

The classifier uses fixed predefined weights and a fixed bias. These values are stored as constants in the VHDL code. Therefore, the FPGA only performs inference. It does not train the model or update the weights during runtime.

The implemented calculation is:

```
z = W0*x0 + W1*x1 + W2*x2 + W3*x3 + B
```

In the current design, the example weights and bias are:

```
W0 = 3
W1 = -2
```

```
W2 = 5
W3 = -1
B  = -4
```

Since the input features are binary, each weight is only added to the sum if the corresponding switch input is active. After the score z is calculated, the activation unit applies a simple threshold:

```
if z >= 0, class_result = 1
else,      class_result = 0
```

This represents a small FPGA-based logistic regression inference system with fixed integer weights and bias.

Output Interface

The `output_interface` module is responsible for displaying the result on the FPGA board. It receives the done signal from the control unit, the score z and the `class_result` signal from the classifier.

When `done = '1'`, the output interface stores the newest score and class result. This is important because the control unit returns to `IDLE` after the operation, but the result should remain visible on board.

The output `CLASS_LED` shows the predicted class. If the classifier result is class 1, the LED becomes active. If the result is class 0, the LED stays inactive.

The seven-segment display shows the absolute value of the calculated score. The score is split into decimal digits and displayed using multiplexing. The `SEG` signals control the segment pattern, and the `AN` signals select the active digit.

Input Signals

CLK:

Main clock signal of the Nexys A7 FPGA board. The design uses the 100 MHz board clock.

RESET:

Reset button input. It clears the control unit, classifier values and output display.

START:

Start button input. It begins one classification cycle.

SW(3 downto 0):

Four FPGA switch inputs are used as binary input features for the classifier.

Output Signals

CLASS_LED:

LED output that shows the predicted class result.

SEG(6 downto 0):

Seven-segment cathode signals used to display the calculated score.

AN(7 downto 0):

Seven-segment anode signals used to select the active display digit.

FPGA Board Implementation

The design was implemented on a Nexys A7 FPGA board. The constraint file connects the VHDL signals to the physical board pins. The four switches SW0 to SW3 are used as input features. The START and RESET signals are connected to push buttons. The CLASS_LED output is connected to LED0. The seven-segment display is controlled using the SEG and AN outputs.

The generated bitstream file `top_level.bit` was loaded onto the FPGA board. Several switch combinations were tested. The test photos show that different input combinations produce different score values on the seven-segment display, such as 5, 6 and 1. This confirms that the classifier output is displayed on the hardware board.

Verification

The `top_level_tb` testbench verifies the complete top-level system. It drives the public input signals `CLK`, `RESET`, `START` and `SW`. It also checks the public output signals `CLASS_LED`, `SEG` and `AN`.

The testbench first checks that the reset state clears the output. Then it verifies that changing the switches without pressing `START` does not update the visible result. After that, it tests all possible 4-bit switch combinations from `"0000"` to `"1111"`. For each input combination, the testbench activates `START` and checks whether a valid seven-segment output is produced.

This verifies that the top-level connection between `control_unit`, `classifier` and `output_interface` works correctly.